

ACTIVIDAD MÓDULO 1

Análisis de Componentes Principales (PCA) con la base Iris

Trabajo realizado en Python mediante Jupyter Notebook / Anaconda

Gael García

Licenciatura en Actuaría

1. Objetivo de la actividad

En esta actividad hicimos un análisis de componentes principales (PCA) con la base Iris usando Python en Jupyter Notebook. La idea fue reducir las cuatro variables numéricas a componentes más fáciles de analizar, ver cuánto explican PC1 y PC2 y después comparar el mapa de observaciones con el mapa de factores.

2. Base de datos utilizada

Usamos el archivo iris.csv. Tiene 150 observaciones y 5 columnas: cuatro variables numéricas y una columna que indica la especie de cada flor.

Variable	Tipo	Descripción
sepal.length	Numérica	Longitud del sépalo
sepal.width	Numérica	Ancho del sépalo
petal.length	Numérica	Longitud del pétalo
petal.width	Numérica	Ancho del pétalo
variety	Categorica	Especie: Setosa, Versicolor o Virginica

Hay 50 flores de cada especie: Setosa, Versicolor y Virginica.

The screenshot shows a Jupyter Notebook window titled "1. Análisis de Componentes Principales - Dropb...". The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. A message at the top states "Matplotlib is building the font cache; this may take a moment." Below this, the code cell [2]: contains the following Python code:

```
df = pd.read_csv("iris.csv")
df
```

The output cell [2]: displays a preview of the DataFrame:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

Below the preview, it indicates "150 rows x 5 columns". The bottom of the notebook shows an empty code cell []: and a large empty output area.

Carga del archivo iris.csv en un DataFrame de Python.

3. Procedimiento realizado en Jupyter Notebook

3.1 Importación de librerías

```
# Tratamiento de datos
import numpy as np
import pandas as pd

# Gráficos
import matplotlib.pyplot as plt

# Preprocesado y modelo
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from numpy import linalg as LA
```

Estas librerías nos sirven para leer los datos, trabajar con ellos, hacer las gráficas y realizar el PCA.

3.2 Carga de la base

```
df = pd.read_csv("iris.csv")
df
```

Cargamos el archivo en un DataFrame llamado df. Al mostrarlo vimos que tiene 150 filas y 5 columnas.

3.3 Selección de variables numéricas

```
datos = df.iloc[:, 0:4]
datos
```

Para el PCA usamos solamente las cuatro variables numéricas. La columna variety la dejamos en df porque después nos ayuda a identificar las especies en las gráficas.

3.4 Estandarización de los datos

```
datos_estandarizados = StandardScaler().fit_transform(datos)
datos_estandarizados
```

Estandarizamos las variables para que todas quedaran en una escala comparable. Por eso aparecen valores positivos y negativos; son valores transformados y no medidas reales de las flores.

3.5 Número de observaciones

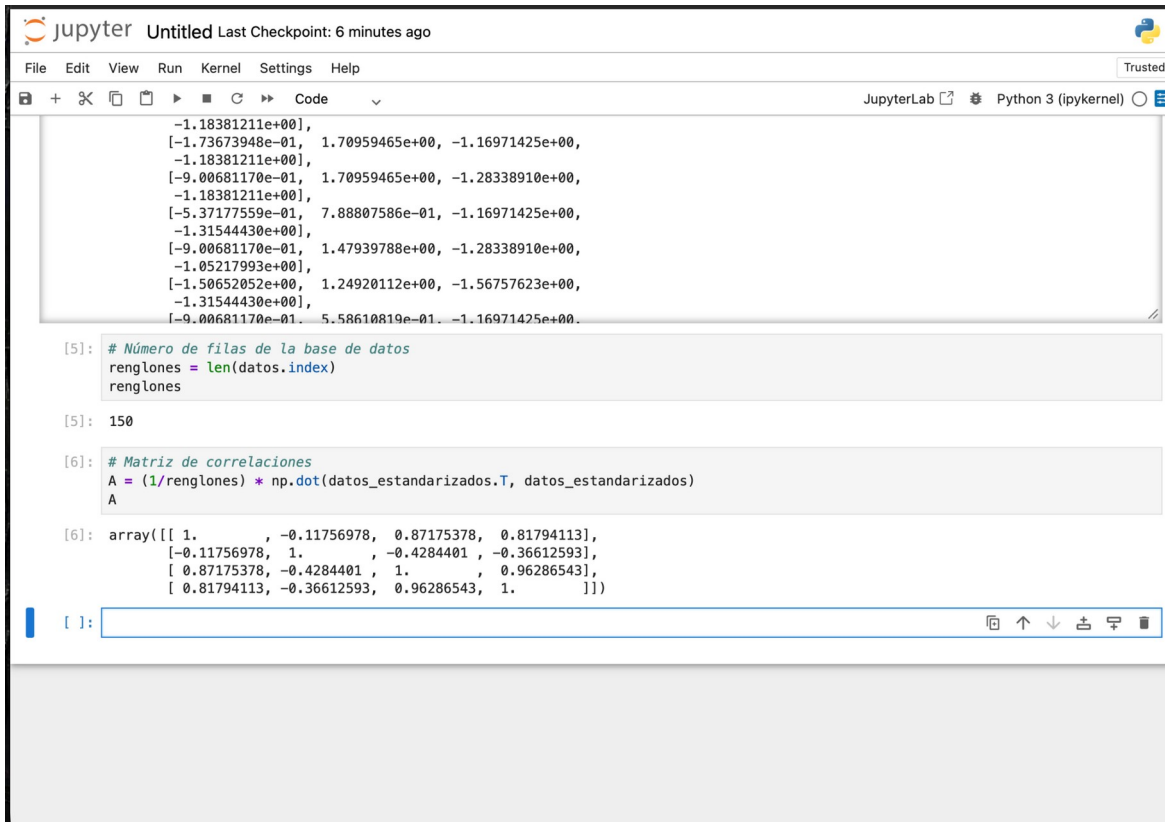
```
# Número de filas de la base de datos
renglones = len(datos.index)
renglones
```

El resultado fue 150, que es el número de flores que estamos usando.

3.6 Matriz de correlaciones

```
# Matriz de correlaciones
A = (1/renglones) * np.dot(datos_estandarizados.T, datos_estandarizados)
A
```

En la matriz se ve que varias variables están relacionadas. La relación más alta es entre petal.length y petal.width, con aproximadamente 0.963. También hay relaciones altas entre sepal.length y petal.length (0.872) y entre sepal.length y petal.width (0.818).



The screenshot shows a JupyterLab window titled 'Untitled' with a last checkpoint of 6 minutes ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations, running code, and kernel management. The code cell contains the following Python code:

```
# Número de filas de la base de datos
renglones = len(datos.index)
renglones

# Matriz de correlaciones
A = (1/renglones) * np.dot(datos_estandarizados.T, datos_estandarizados)
A
```

The output of the code cell shows the value of 'renglones' as 150 and the correlation matrix 'A' as a 4x4 array:

```
array([[ 1.          , -0.11756978,  0.87175378,  0.81794113],
       [-0.11756978,  1.          , -0.4284401 , -0.36612593],
       [ 0.87175378, -0.4284401 ,  1.          ,  0.96286543],
       [ 0.81794113, -0.36612593,  0.96286543,  1.          ]])
```

Matriz de correlaciones calculada con los datos estandarizados.

3.7 Modelo PCA y eigenvalores

```
# Modelo PCA
pca_pipe = make_pipeline(StandardScaler(), PCA())
pca_pipe.fit(datos)

# Extraemos el modelo PCA
modelo_pca = pca_pipe.named_steps['pca']

# Eigenvalores
print("Eigenvalores:")
resultados = LA.eigvals(A)
print(resultados)
```

Los eigenvalores fueron aproximadamente 2.9185, 0.9140, 0.1468 y 0.0207. Los dos primeros son los más grandes, así que desde aquí se nota que PC1 y PC2 guardan la mayor parte de la información.

3.8 Porcentaje de varianza explicada

```
print("Porcentaje de varianza explicada por componente:")
print(modelo_pca.explained_variance_ratio_)
```

Componente	Proporción	Porcentaje
PC1	0.72962445	72.96 %
PC2	0.22850762	22.85 %
PC3	0.03668922	3.67 %
PC4	0.00517871	0.52 %

PC1 + PC2 = 72.96 % + 22.85 % = 95.81 %

Entonces, PC1 y PC2 juntos explican aproximadamente el 95.81 % de la variación. Es un porcentaje muy alto, así que con esos dos componentes podemos representar casi toda la información de las cuatro variables numéricas.

```
[8]: # Porcentaje de varianza explicada por cada componente
print("Porcentaje de varianza explicada por componente:")
print(modelo_pca.explained_variance_ratio_)

Porcentaje de varianza explicada por componente:
[0.72962445 0.22850762 0.03668922 0.00517871]
```

```
[ ]: |
```

Resultado del porcentaje de varianza explicada por cada componente.

3.9 Eigenvectores

```
# Eigenvectores
print("Eigenvectores:")

pd.DataFrame(
    data=modelo_pca.components_,
    columns=datos.columns,
    index=['PC1', 'PC2', 'PC3', 'PC4']
)
```

En PC1 destacan petal.length, petal.width y sepal.length. En PC2 destaca sobre todo sepal.width. Esto nos ayuda a entender qué variables influyen más en cada componente.

```
[9]: # Eigenvectores
print("Eigenvectores:")

pd.DataFrame(
    data=modelo_pca.components_,
    columns=datos.columns,
    index=['PC1', 'PC2', 'PC3', 'PC4']
)

Eigenvectores:
[9]:
```

	sepal.length	sepal.width	petal.length	petal.width
PC1	0.521066	-0.269347	0.580413	0.564857
PC2	0.377418	0.923296	0.024492	0.066942
PC3	0.719566	-0.244382	-0.142126	-0.634273
PC4	-0.261286	0.123510	0.801449	-0.523597

```
[ ]: |
```

Eigenvectores del PCA.

3.10 Proyecciones de las observaciones

```
# Proyecciones de los componentes
proyecciones = np.dot(modelo_pca.components_, datos_estandarizados.T)

proyecciones = pd.DataFrame(
    proyecciones,
    index=['PC1', 'PC2', 'PC3', 'PC4']
)
```

```
proyecciones = proyecciones.transpose()
proyecciones
```

Aquí obtuvimos la posición de cada una de las 150 flores en PC1, PC2, PC3 y PC4. Para las gráficas usamos PC1 y PC2.

4. Mapa de observaciones

```
# Mapa de observaciones
plt.figure(figsize=(8,6))

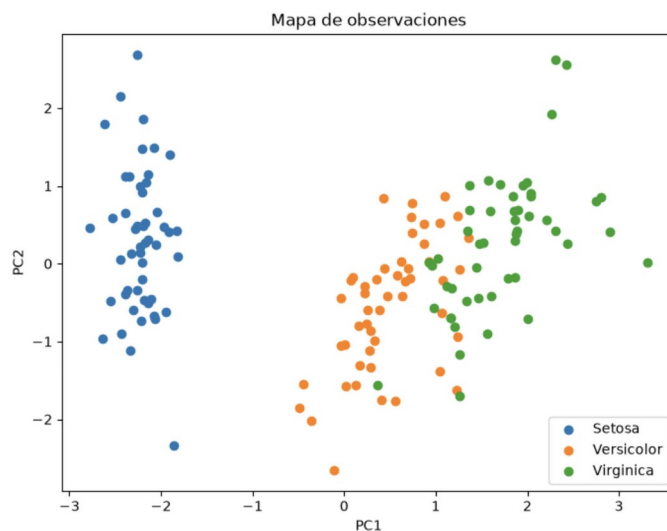
for especie in df['variety'].unique():
    grupo = proyecciones[df['variety'] == especie]
    plt.scatter(grupo['PC1'], grupo['PC2'], label=especie)

plt.title("Mapa de observaciones")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.show()
```

```
[11]: # Mapa de observaciones
plt.figure(figsize=(8,6))

for especie in df['variety'].unique():
    grupo = proyecciones[df['variety'] == especie]
    plt.scatter(grupo['PC1'], grupo['PC2'], label=especie)

plt.title("Mapa de observaciones")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.show()
```



Mapa de observaciones en PC1 y PC2, separado por especie.

- Setosa aparece claramente separada de las otras dos especies.
- Versicolor y Virginica se encuentran más cerca entre sí y presentan cierto traslape.
- Virginica tiende a ubicarse en valores más positivos de PC1, mientras que Setosa aparece en valores negativos de PC1.

Como PC1 y PC2 explican el 95.81 % de la variación, esta gráfica nos da una buena idea de cómo se acomodan las especies.

5. Mapa de factores

5.1 Preparación de los factores

```
# Datos para el mapa de factores
componentes_2 = pd.DataFrame(
    data=modelo_pca.components_,
    columns=datos.columns,
    index=['PC1', 'PC2', 'PC3', 'PC4']
)

componentes_2 = componentes_2.iloc[0:2, :]
componentes_2 = componentes_2.T
componentes_2
```

Tomamos solamente PC1 y PC2 y acomodamos las cuatro variables para ver cómo se relacionan con estos dos componentes.

5.2 Gráfica del mapa de factores

```
# Mapa de factores
x = componentes_2.iloc[:,0]
y = componentes_2.iloc[:,1]
z = componentes_2.index

x = x.to_numpy()
y = y.to_numpy()

fig, ax = plt.subplots()
ax.set_title("Mapa de factores")
ax.scatter(x, y)

for i, txt in enumerate(z):
    ax.annotate(txt, (x[i], y[i]))

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

```
[13]: # Mapa de factores
x = componentes_2.iloc[:,0]
y = componentes_2.iloc[:,1]
z = componentes_2.index

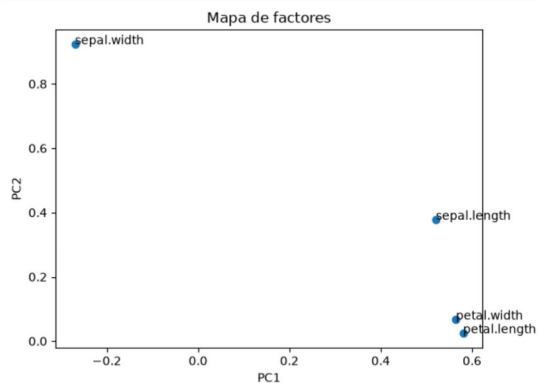
x = x.to_numpy()
y = y.to_numpy()

fig, ax = plt.subplots()

ax.set_title("Mapa de factores")
ax.scatter(x, y)

for i, txt in enumerate(z):
    ax.annotate(txt, (x[i], y[i]))

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```



Mapa de factores de las cuatro variables originales.

- petal.length y petal.width aparecen muy cerca, así que tienen un comportamiento parecido.
- petal.length y petal.width están fuertemente relacionadas con PC1.
- sepal.length también se relaciona con PC1, aunque en menor medida.
- sepal.width es la variable que más se relaciona con PC2.

6. Comparación de los mapas y comprobación con casos específicos

Al comparar los dos mapas vemos que el tamaño del pétalo ayuda mucho a separar las especies. PC1 está muy relacionado con petal.length y petal.width, y también es el eje donde Setosa se separa con más claridad de las otras dos especies.

```
# Casos específicos del DataFrame
df.loc[[0, 50, 100]]
```

```
[14]: # Casos específicos del DataFrame
df.loc[[0, 50, 100]]
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
50	7.0	3.2	4.7	1.4	Versicolor
100	6.3	3.3	6.0	2.5	Virginica

Casos específicos utilizados para comprobar la interpretación.

Estos tres casos coinciden con lo que vimos en las gráficas: Setosa tiene pétalos pequeños, Versicolor queda en valores intermedios y Virginica tiene pétalos más grandes. Por eso el tamaño del pétalo ayuda a separar las especies.

7. Conclusiones finales

- PC1 y PC2 juntos explican el 95.81 % de la variación. PC1 explica el 72.96 % y se relaciona principalmente con el tamaño del pétalo; PC2 explica el 22.85 % y se relaciona sobre todo con el ancho del sépalo.
- Setosa forma un grupo claramente separado; Versicolor y Virginica presentan mayor cercanía y cierto traslape.
- En general, el tamaño del pétalo es una de las características que más ayuda a separar las especies en los dos primeros componentes.